

Lecture – 5

Strings

Creating and Accessing String

- The string literal (hello) is enclosed in single quotation marks (') or double quotation marks (").
- \$a = "hello"; \$b = 'hello'; // \$a and \$b are the same
- In order to access to individual characters within a string, use curly brace { } with index number starting from 0. (The first character is {0})

Example:

```
$myst="Hello"; print $myst. "<pre>\n</pre>". 'World';  
$favoriteAnimal = "cat";  
echo "My favorite animals are {$favoriteAnimal}s"; //Display cats
```

- String indices start from 0.
//Access character in a string
echo \$myst[0] . " < br / > "; //Display "H"

Escape sequences

Sequence	Meaning
<code>\n</code>	A line feed character (ASCII 10)
<code>\r</code>	A carriage return character (ASCII 13)
<code>\t</code>	A horizontal tab character (ASCII 9)
<code>\v</code>	A vertical tab character (ASCII 11)
<code>\f</code>	A form feed character (ASCII 12)
<code>\\</code>	A backslash (as opposed to the start of an escape sequence)
<code>\\$</code>	A \$ symbol (as opposed to the start of a variable name)
<code>\”</code>	A double quote (as opposed to the double quote marking the end of a string)

Examples

- `// Output with \n for command-line usage or terminal output \n creates new line`
- `echo "Hello, World!\n";`
- `echo "This is a new line.\n";`
- `// Using \n inside a string`
- `$message = "Line 1\nLine 2\nLine 3";`
- `echo $message;`

\r in Browser Output

```
echo "Hello, World!\rPHP"; // Hello, World!PHP
```

```
// PHPlo, World! ( Use command line)
```

\t

```
<!DOCTYPE html>
<html>
<body>

<?php
// Browser θα εμφανίσει \t output
echo "Name\tAge\tCountry";
?>

</body>
</html>
```

String Functions

- `strlen(string variable)` //finding the length of a string
- `str_word_count(string variable)` //returns the number of words in a string
- `strstr()` //tells you whether the search text is within the string
- `strpos()` and `strrpos()` //return the index position of the first and last occurrence of the search text
- `substr_count()` //tells you how many times the search text occurs within the string
- `strpbrk()` //searches a string for any of a list of characters

Example:

```
$myString = "hello world!";  
echo strlen( $myString ) . "<br/>"; // Displays 12  
echo str_word_count( "Hello, world!" ); // Displays 2  
echo strstr( $myString, "wor" ) . "<b/>"; // Displays 'world!'  
echo ( strstr( $myString, "yz" ) ? "Yes" : "No" ) . "<b/>"; //Display No
```

Accessing Characters within a String

- substr() - to extract a sequence of characters from a string
 - If you use a negative number, substr() counts backward from the end of the string.

Example1:

```
$myString = "Hello, world!";
```

```
echo substr( $myString, 0, 5 ) . " < br/ > "; // Displays 'Hello'
```

```
echo substr( $myString, 7 ) . " < br/ > "; // Displays 'world!'
```

```
echo substr( $myString, -1 ) . " < br/ > "; // Displays '!'
```

```
echo substr( $myString, -5, -1 ) . " < br/ > "; // Displays 'orld'
```

Replacing Text within Strings

- `str_replace()` replaces all occurrences of the search text within the target string
- `substr_replace()` replaces a specified portion of the target string with another string
- `strtr()` replaces certain characters in the target string with other characters

Example

```
$myString = "It was the best of times, it was the worst of times,";
```

```
echo str_replace( "times", "bananas", $myString );
```

```
// Displays "It was the best of bananas, it was the worst of bananas,"
```

```
echo substr_replace( $myString, "bananas", 11 ) . " < br/ > ";
```

```
// Displays "It was the bananas"
```

For strtr():

```
$myString = "Here's a little string";
```

```
echo strtr( $myString, " ' ", "+" ) . " < br/ > ";
```

```
// Displays "Here+s a little string"
```

Example:

```
$trans = array("h" => "-", "hello" => "hi", "hi" => "hello");
```

```
echo strtr("hi all, I said hello", $trans);
```

```
//Display "hello all, I said hi"
```

Dealing with Upper - and Lowercase

- To convert a string to all lowercase, use **strtolower()** .
`$myString = "Hello, world!";`
`echo strtolower( $myString );` // Displays 'hello, world!'
- Similarly, you can use **strtoupper()** to convert a string to all uppercase.
`echo strtoupper( $myString );` // Displays 'HELLO, WORLD!'
- **ucfirst()** makes just the first letter of a string uppercase.
`echo ucfirst( $myString );` // Displays 'Hello, world!'
- **lcfirst()** — introduced in PHP 5.3 — makes the first letter of a string lowercase.
`echo lcfirst($myString);` // Displays 'hello, world!'
- **ucwords()** makes the first letter of each word in a string uppercase.
`echo ucwords( $myString );` // Displays 'Hello, World!'

Formatting String

- to format strings in ways that are more human friendly
- more common string formatting functions in PHP - printf () and sprintf()

Example:

```
printf( "<br>Pi rounded to a whole number is: %d", M_PI );  
$myNumber = 123.45;  
printf( "<br/>Binary: %b <br/>", $myNumber );  
printf( "Character: %c <br/>", $myNumber );  
printf( "Decimal: %d <br/>", $myNumber );  
printf( "Scientific: %e <br/>", $myNumber );  
printf( "Float: %f<br/>", $myNumber );  
printf( "Octal: %o <br/>", $myNumber );  
printf( "String: %s <br/>", $myNumber );  
printf( "Hex (lower case): %x <br/>", $myNumber );  
printf( "Hex (upper case): %X <br/>", $myNumber );
```

Trimming String

- Remove the unwanted white space at the beginning or end of the text (or both)

Example:

```
$myString = " What a lot of space! ";  
echo "<pre>";  
echo "|" . trim( $myString ) . "\\n";  
// Displays "|What a lot of space!|"   
echo "|" . ltrim( $myString ) . "\\n";  
// Displays "|What a lot of space! |";  
echo "|" . rtrim( $myString ) . "\\n";  
// Displays "| What a lot of space!|";  
echo "</pre>";
```

Formatting Numbers

- `number_format()` function gives to format numbers in an easy to read way

Example:

```
echo "<br/>".number_format( 1234567.89 );
```

```
// Displays “1,234,568”
```

```
echo "<br/>".number_format( 1234567.89 ,1);
```

```
//Displays “1,234,567.9”
```

```
echo "<br/>".number_format( 1234567.89, 2, ",", " " );
```

```
//Displays “1 234 567,89”
```

Exercises

1. Write a script that emulates the function call `str_pad( $myString, $desiredLength)`. In other words, take a string, and add space characters to the right of it until the string reaches the desired length. Display both the original and padded string in the page.